Московский авиационный институт
(национальный исследовательский университет)

Факультет компьютерных наук и прикладной математики

Кафедра вычислительной математики и программирования

Курсовая работа по курсу «Дискретный анализ»

Сжатие данных методом LZW

Студент: Волков А.А.
Преподаватель:
Группа: М8О-301БВ-24
Дата: 11 ноября 2026 г.
Оценка:
Подпись:

Москва, 2026

Курсовая работа. Сжатие данных методом LZW

Задача: Разработать кодирование и декодирование строк методом LZW. Алфавит состоит из строчных латинских букв. Начальные коды: a — 0, b — 1, . . . , z — 25. Код EOF равен 26, новые записи начинаются с 27. В режиме compress вводится строка, выводятся числовые коды. В режиме decompress вводятся коды до EOF, выводится исходная строка. Например, abracadabra соответствует последовательности 0 1 17 0 2 0 3 27 29 26.

1 Описание

1 Основная идея решения

Алгоритм LZW выполняет сжатие без потерь, заменяя подстроки исходного текста числовыми кодами. Соответствие между подстроками и кодами хранится в словаре, который пополняется во время обработки данных.

Первоначально словарь содержит строчные латинские буквы с кодами от 0 до 25. Код 26 обозначает конец данных — EOF. Новым подстрокам присваиваются коды начиная с 27.

При кодировании алгоритм ищет наиболее длинную подстроку, которая уже присутствует в словаре. В результат записывается её код, а эта подстрока вместе со следующим символом добавляется в словарь. Благодаря этому повторно встретившиеся сочетания символов могут представляться одним кодом.

Декодировщик начинает с такого же начального словаря и восстанавливает новые записи по мере чтения кодов. [1].

2 Подробное описание работы

Кодирование

Для кодирования используется словарь `std::unordered_map`, в котором ключом является подстрока типа `std::string`, а значением — её целочисленный код. Переменная `currentString` хранит текущую найденную подстроку, а `nextCode` — следующий свободный код.

Рабочая строка первоначально пуста. Входная строка последовательно обрабатывается по символам:

1. К рабочей строке присоединяется очередной символ. Полученная строка рассматривается как кандидат на расширение.

2. Если кандидат уже присутствует в словаре, он становится рабочей строкой.

3. Если кандидат отсутствует, код рабочей строки записывается в результат, кандидат добавляется в словарь под следующим свободным кодом. Рабочей строкой становится последний прочитанный символ.

На первой итерации кандидат состоит из одной буквы и уже присутствует в начальном словаре. Поэтому первый символ только записывается в рабочую строку. После завершения цикла в результат добавляется код оставшейся рабочей строки, а затем код EOF.

Декодирование

Для декодирования используется обратный словарь: ключом является целочисленный код, а значением — соответствующая ему строка.

Первый код обрабатывается отдельно. Соответствующая ему строка записывается в результат и становится рабочей строкой. Если входная последовательность пуста или сразу начинается с EOF, то результат — пустая строка.

Для каждого следующего кода выполняются действия:

1. Если прочитан EOF, обработка завершается.

2. Проверяется наличие кода в словаре:

   (a) Если код существует, новая строка получается из словаря по этому коду.

   (b) Если код отсутствует, новая строка составляется из рабочей строки и её первого символа. Для корректного входа это специальный случай декодирования, описанный ниже.

3. Формируется новая запись словаря: рабочая строка вместе с первым символом новой строки. Полученная запись добавляется под следующим свободным кодом, после чего этот код увеличивается на единицу.

4. Новая строка становится рабочей и добавляется в результат.

Например, при чтении кодов 0, 1, 17 декодер получает строки a, b, r. После получения b создаётся запись $27 \to$ `ab`, а после получения r — запись $28 \to$ `br`. Таким образом, необходимые записи восстанавливаются по ходу обработки.

Специальный случай декодирования

В корректной последовательности может встретиться код, которого ещё нет в словаре декодера. Это происходит, когда кодировщик создал новую запись и сразу использовал её. Такой код равен следующему свободному коду декодера.

Неизвестная запись должна состоять из предыдущей строки и первого символа текущей строки. При этом текущая строка соответствует новой строке. Следовательно, она начинается с предыдущей строки, и их первые символы совпадают. Поэтому её можно восстановить как предыдущую строку с добавленным её первым символом.

Например, строка aaa кодируется последовательностью 0 27 26. После чтения 0 декодер выводит a. Следующий код 27 пока отсутствует в словаре. Его строка восстанавливается как `a` + `a` = `aa`. Декодер добавляет запись 27 → `aa` и дописывает aa к результату. Получается исходная строка aaa.

Оценка сложности

Для кодирования n — длина входной строки, а L — максимальная длина проверяемой строки. Выполняется n итераций, но создание, хеширование и копирование кандидата требуют до $O(L)$ времени. Получаем верхнюю оценку $O(nL)$.

Декодирование выполняется за $O(N)$, где N — длина восстановленной строки. Добавление символов к результату учитывается амортизированно.

2 Тест производительности

Для сравнения выбран DEFLATE из библиотеки zlib: он сочетает поиск повторов LZ77 и коды Хаффмана и служит готовым библиотечным ориентиром [3, 4]. Второй метод, RLE, — распространённое кодирование серий одинаковых символов парой «символ, количество повторений» [2]. Он позволяет сравнить словарное сжатие с подходом, эффективным на длинных сериях.

Команда сборки:

```
g++ -std=c++20 -O2 benchmark.cpp -lz -o benchmark
```

Запуск: `./benchmark 5 > results.csv`.

Для каждой пары размера и типа данных выполнен прогрев и пять повторений. Приведена медиана отдельно для сжатия и восстановления. Создание словарей, выделение выходных буферов, сериализация и восстановление входят в замеры. Генерация, консольный вывод и проверка равенства исходной и восстановленной строк не входят.

Использованы следующие типы данных:

- `constant` — строка из одинаковых букв a;
- `periodic` — повторение abracadabra с обрезкой до нужной длины;
- `random` — независимый равномерный выбор из 26 строчных латинских букв.

Для случайных строк используется генератор псевдослучайных чисел `mt19937` с одинаковым начальном сидом. Для каждого размера генератор инициализируется заново.

Форматы. LZW хранит коды в битовом потоке с переменной шириной. Начальная ширина составляет 5 бит. Перед очередным кодом она увеличивается на единицу, если следующий свободный код восстанавливаемого словаря уже не помещается в прежнее количество бит. Упаковщик и распаковщик используют одинаковое правило, учитывающее, что после первого кода декодер ещё не добавляет запись в словарь. После переключения все коды, включая EOF, записываются с новой шириной. Неиспользованные биты последнего байта заполняются нулями. Упаковка и распаковка входят в измеряемое время. RLE хранит символ одним байтом и длину серии четырьмя байтами. DEFLATE измеряется в оболочке zlib, включая её заголовок и контрольную сумму. Сжатие выполняется `compress2` с заданным уровнем, восстановление — `uncompress` [4]. Исходный размер передаётся декодерам отдельно; размер контейнера с этим метаданным не учитывается ни для одного метода. Сравнивается размер полезного двоичного представления, а не текстовых чисел.

Отношение $R = S_{\text{encoded}}/S_{\text{input}}$ меньше единицы означает сжатие. В таблице времена указаны в миллисекундах, размеры — в байтах. Порядок методов фиксирован.

Тип	Вход	Метод	Выход	R	Сжатие	Восст.
constant	1000	DEFLATE	17	0.017000	0.005	0.002
constant	1000	LZW	35	0.035000	0.030	0.007
constant	1000	RLE	5	0.005000	0.001	0.001
periodic	1000	DEFLATE	26	0.026000	0.039	0.002
periodic	1000	LZW	120	0.120000	0.031	0.011
periodic	1000	RLE	4550	4.550000	0.006	0.011
random	1000	DEFLATE	629	0.629000	0.107	0.012
random	1000	LZW	794	0.794000	0.065	0.043
random	1000	RLE	4790	4.790000	0.005	0.005
constant	10000	DEFLATE	34	0.003400	0.053	0.019
constant	10000	LZW	124	0.012400	0.332	0.015
constant	10000	RLE	5	0.000500	0.005	0.001
periodic	10000	DEFLATE	51	0.005100	0.053	0.007
periodic	10000	LZW	470	0.047000	0.312	0.046
periodic	10000	RLE	45455	4.545500	0.038	0.038
random	10000	DEFLATE	6255	0.625500	0.137	0.027
random	10000	LZW	7270	0.727000	0.604	0.343
random	10000	RLE	48065	4.806500	0.038	0.038
constant	100000	DEFLATE	121	0.001210	0.261	0.171
constant	100000	LZW	457	0.004570	5.110	0.083
constant	100000	RLE	5	0.000050	0.038	0.002
periodic	100000	DEFLATE	226	0.002260	0.291	0.056
periodic	100000	LZW	1793	0.017930	3.936	0.163
periodic	100000	RLE	454550	4.545500	0.515	0.305

Тип	Вход	Метод	Выход	R	Сжатие	Восст.
random	100000	DEFLATE	63454	0.634540	1.731	0.330
random	100000	LZW	69320	0.693200	5.815	2.363
random	100000	RLE	480775	4.807750	0.342	0.329
constant	1000000	DEFLATE	992	0.000992	2.481	1.646
constant	1000000	LZW	1714	0.001714	98.953	0.519
constant	1000000	RLE	5	0.000005	0.329	0.011
periodic	1000000	DEFLATE	1969	0.001969	2.535	0.531
periodic	1000000	LZW	6611	0.006611	61.210	0.630
periodic	1000000	RLE	4545455	4.545455	4.421	3.536
random	1000000	DEFLATE	635630	0.635630	19.093	3.473
random	1000000	LZW	678848	0.678848	114.929	30.739
random	1000000	RLE	4807365	4.807365	3.511	3.326

3 Выводы

Под наилучшим результатом по степени сжатия далее понимается минимальный размер закодированного представления. Скорость рассматривается отдельно.

1. На наборе одинаковых символов наилучшим оказался RLE: вся строка представляется одной серией. Он также показал наименьшее время сжатия и восстановления на всех проверенных размерах.

2. На случайных строках наилучший размер показал DEFLATE. Алфавит ограничен строчными латинскими буквами, поэтому даже при независимом выборе символов их восьмибитное хранение содержит избыточность, которую использует кодирование Хаффмана. RLE быстрее сжимает эти данные, но увеличивает их размер. LZW сокращает размер всех проверенных случайных наборов, хотя уступает DEFLATE по компактности результата.

3. На периодических строках наилучший размер показал DEFLATE: повторы блоков подходят для ссылок LZ77. LZW также сокращает размер, а RLE не использует повторение целых блоков и расширяет данные.

Применимость LZW

LZW целесообразно рассматривать для данных с повторяющимися подстроками, включая структурированные текстовые записи и повторяющиеся блоки. Преимущество особенно заметно, если требуется уменьшить размер периодических данных по сравнению с простым кодированием серий.

Для сравнения размеров используется процент $100(S_{\mathrm{LZW}} - S_{\mathrm{base}})/S_{\mathrm{base}}$: отрицательное значение означает уменьшение относительно метода сравнения. Все следующие диапазоны получены из попарных сравнений на одинаковых входах для четырёх проверенных размеров.

На периодических данных представление LZW меньше RLE на 97.36–99.85%, но больше DEFLATE на 235.75–821.57%. В данной реализации выигрыш в размере относительно RLE оплачивается более длительным кодированием.

На случайных строках LZW занимает на 83.42–85.88% меньше места, чем RLE, но на 6.80–26.23% больше, чем DEFLATE. При этом размер результата LZW меньше исходного на 20.60–32.12%. Для длинных серий одного символа предпочтительнее RLE; LZW здесь не даёт выигрыша в размере или скорости.

При увеличении длины периодического набора от 1000 до 1000000 символов доля размера LZW относительно исходного уменьшается с 12.00% до 0.6611%. Для случайного набора той же длины доля уменьшается с 79.40% до 67.8848%. На периоди-

ческих данных увеличение длины даёт более выраженное снижение доли выходного размера, чем на случайных буквах.

Итого Изучил принцип словарного сжатия LZW и восстановление словаря при декодировании. На практике убедился, что эффективность сжатия зависит не только от алгоритма, но и от способа хранения его результата. Сравнение LZW с другими алгоритмами сжатия показало, что выбор метода зависит от структуры входных данных и требований к скорости и размеру результата.

Список литературы

[1] Викиконспекты ИТМО. Алгоритм LZW. Статья об алгоритме LZW.

[2] Викиконспекты ИТМО. Алгоритм RLE. Статья об алгоритме RLE.

[3] Deutsch P. DEFLATE Compressed Data Format Specification version 1.3. RFC 1951, 1996. `https://www.rfc-editor.org/rfc/rfc1951`.

[4] Gailly J.-l., Adler M. zlib Manual. `https://zlib.net/manual.html`.